Two-Alternation Reconfigurable Routing on Hypercubes is NP-Complete

Research manuscript · awaiting expert review
16 September 2026

Abstract

A reconfigurable network augments a static graph with dynamic links selected by an optical switch. We study the minimum-workload decision problem when the static graph is a hypercube, every vertex has one switch port, static links cost one, and dynamic links cost one half. We give a deterministic polynomial-time reduction from Partial Domination on hypercubes when each route may alternate between static and dynamic links at most twice. The construction occupies a ball around the destination's forced partner, excluding the additional two-dynamic-edge routes that distinguish this problem from the known one-dynamic-edge case. Every feasible target matching yields a source dominating set in polynomial time. The reduction has polynomial size even when the target cube is written explicitly.

1. Introduction

Reconfigurable routing separates a fixed network topology from the dynamic links chosen to serve a workload. Complexity results for structured static networks therefore address a different restriction from hardness on arbitrary weighted graphs. Hypercubes provide a standard family with uniform static link costs and an explicit distance formula.

Kutner and Stewart establish hardness on hypercubes with three medium alternations, and separately with at most one dynamic edge per route [1]. Their full preprint leaves two alternations open. These restrictions are not interchangeable: a route with two alternations may have the form dynamic–static–dynamic. Allowing this additional shape can introduce feasible matchings for source NO instances.

We extend the domination-and-slot construction underlying the one-dynamic-edge result. The added component occupies a neighborhood of the destination's forced matching partner. It prevents a source vertex from acquiring a sufficiently close partner for a cheap two-dynamic-edge route. The remainder of the reduction uses a tight workload threshold to recover a dominating set.

Theorem 1. One-switch reconfigurable routing on hypercubes is NP-complete when every vertex has one port, static links cost 1, dynamic links cost $\frac{1}{2}$, and routes have at most two medium alternations. There is a deterministic polynomial-time Karp reduction from Partial Domination on explicitly represented hypercubes, together with a polynomial-time map recovering a dominating set from every feasible matching of the constructed instance.

The theorem applies to the usual explicit-network representation. Our executable map writes a unary cube dimension and sparse demands; we separately bound the cost of expanding that description. The result does not address one alternation or supply an efficient algorithm for finding a target configuration.

2. Decision problems and routes

Write Q_d for the graph on binary strings of length d, with edges joining strings that differ in one coordinate. Let $D(u, v)$ denote Hamming distance, and let $B(c, j)$ be the closed ball of radius j around c.

A source instance consists of $d \geq 1$, a set $T \subseteq V(Q_d)$ given by its full activity vector, and an integer $k \geq 0$. It is a YES instance if some $X \subseteq T$ with $|X| \leq k$ dominates the induced graph $Q_{d[T]}$. This source is NP-complete by Theorem 7 of the full preprint of [1]. Its proof constructs coordinates in a cube of polynomial size, so the explicit activity encoding used here preserves that hardness.

A target configuration is a partial matching P on the cube vertices. Each matching edge is a dynamic link of cost $\frac{1}{2}$; each cube edge is a static link of cost 1. A positive integer demand weight multiplies the minimum path cost for its ordered endpoints. There are no capacity constraints. The question asks

whether the total workload is at most a specified threshold. Every path has at most two changes of link medium. Static edges remain available when a dynamic edge has the same endpoints.

Let m be the partial mate function of P. In doubled cost units, the shortest permitted route from u to v has length

$$\min \begin{cases} 2D(u,v) & \\ 1 + 2D(u,a) + 2D(b,v) & \{a,b\} \in P \text{ in either orientation} \\ 2 + 2D(m(u), m(v)) & \text{when both mates exist.} \end{cases} \tag{1}$$

A simple path cannot traverse two distinct dynamic edges consecutively, because P is a matching. Its possible shapes are consequently static, static–dynamic–static, and dynamic–static–dynamic, allowing empty static segments. Conversely, the expressions in Equation 1 specify walks of these shapes. Deleting a cycle cannot increase cost or the number of medium changes, so the same minimum is attained by a permitted path.

The target is in NP even with a unary dimension and sparse demands. From a feasible configuration retain only the dynamic edges used by one chosen route for each demand. At most two edges per demand remain, and those routes are preserved. Equation 1 checks the resulting polynomial-size certificate using exact integer arithmetic.

3. Construction

We may assume $1 \le k < n$, where $n = |T|$. An empty active set or $k \ge n$ maps to a fixed YES instance and recovers T. A nonempty set with $k = 0$, or a malformed source encoding, maps to a fixed NO instance.

Set

$$r = d, \quad b = 2d + 3, \quad \ell = 4b + d = 9d + 12. \tag{2}$$

Embed Q_d in the low d coordinates of Q_ℓ and set the remaining coordinates to zero; call this embedded cube M. Divide the remaining coordinates into four blocks of size b. Let o, s, g, h be their respective all-one masks, zero outside the corresponding block. We use xor for bitwise exclusive-or.

Let L contain the first k integer-labeled vertices of the sphere of radius r around o. This sphere has at least 2^d vertices: choose one coordinate from each of d disjoint coordinate pairs. The slots therefore exist for every nontrivial source instance. Let $N[M]$ denote the closed neighborhood of M and set

$$F = (N[M] \cup B(o, r+1)) \setminus (T \cup L). \tag{3}$$

The following edge families form a matching E:

$$\begin{aligned} E = {} & \{\{v, v \operatorname{xor} h\} : v \in F \setminus \{o\}\} \\ & \cup \{\{o, s\}\} \\ & \cup \{\{v, v \operatorname{xor} g\} : v \in B(s, r) \setminus \{s\}\}. \end{aligned} \tag{4}$$

Write $q = |E|$. Give each pair in E a demand of weight $\alpha = n(r+2) + 1$, and give each pair (u, o) with $u \in T$ a unit demand. The doubled workload bound is

$$K = \alpha q + n(2r + 3) - 2k. \tag{5}$$

The output specifies these demands, not a fixed configuration. Their weights will force every feasible configuration to contain E.

Figure 1 shows how a selected vertex uses a slot and why the partner ball is needed for soundness. Paths to the destination are aggregated in the drawing; the constructed network itself remains an unmodified hypercube.

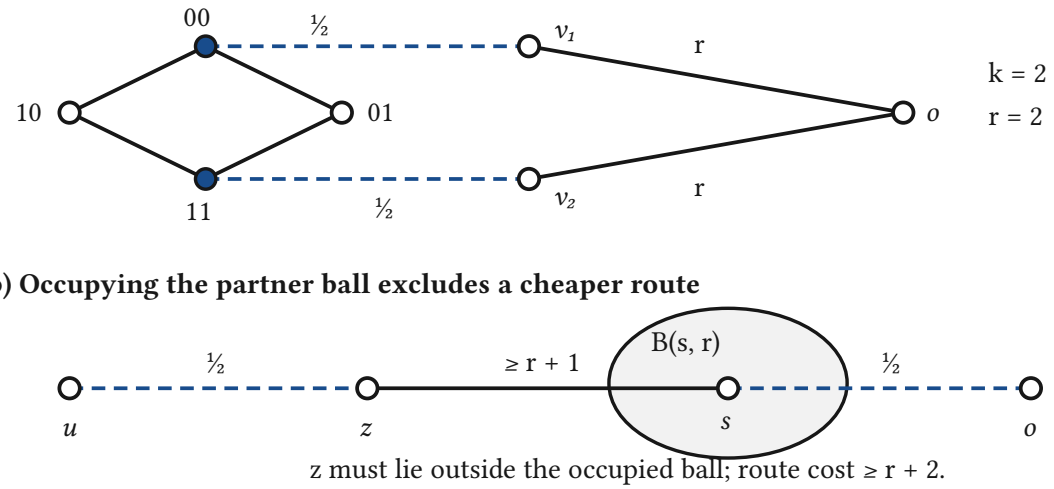


Figure 1: The reduction mechanism, shown schematically. In (a), the four source vertices form an exact Q_2; filled vertices 00 and 11 form a dominating set. Dashed links are dynamic matching edges, of cost $\frac{1}{2}$. Solid source edges cost one; the solid slot-to-o segments represent static paths of length r, not single edges. In (b), the final dynamic edge is the forced pair $\{s, o\}$. Every vertex of $B(s, r)$ is already matched outside T, so the mate z of a source vertex u must lie outside this ball. The intervening static path has length at least $r + 1$. Other forced pairs and cube edges are omitted; positions are not a metric embedding.

3.1. Separation of the forced pairs

Lemma 2. The edge set E is a matching, disjoint from $T \cup L$. It occupies all of $B(s, r)$, all of $B(o, r + 1) \setminus L$, and all of $N[M] \setminus T$.

Proof. The six regions containing its endpoints have centers 0, o, h, $o \operatorname{xor} h$, s, $s \operatorname{xor} g$ and radii at most $r + 1$. Distinct centers differ in at least one entire block, so their distances are at least $b = 2r + 3$. Their balls are disjoint. Within each region, exclusive-or translation is injective. Excluding o and s from their translation families avoids conflicts with the special pair $\{o, s\}$. The subtractions in Equation 3 leave every source vertex and slot unused. The stated occupancy follows. □

For later use, every $u \in T$ has distance at least b from both o and s. If $v \in B(o, r + 1) \setminus (L \cup \{o\})$, its forced mate obeys

$$D(u, v \operatorname{xor} h) \geq 2b - (r + 1). \tag{6}$$

Indeed, u is at distance at least $2b$ from $o \operatorname{xor} h$, while the translated v lies within $r + 1$ of that center. Every vertex in $B(s, r)$ has its forced mate outside T: the mate of s is o, and all other mates lie in the separated ball around $s \operatorname{xor} g$.

4. Correctness and recovery

4.1. Completeness

Let $X \subseteq T$ dominate T with $|X| \leq k$. Pad it within T to size k and match its vertices bijectively to L, in addition to E. Lemma 2 ensures that the union is a legal matching. Each selected vertex reaches o through its slot at cost $r + \frac{1}{2}$. Each unselected vertex is adjacent to a selected one and therefore has a route of cost at most $r + \frac{3}{2}$. Forced demands use their direct links. The workload is at most

$$\frac{\alpha q}{2} + k\left(r + \frac{1}{2}\right) + (n - k)\left(r + \frac{3}{2}\right) = \frac{K}{2}. \tag{7}$$

4.2. Soundness for arbitrary feasible matchings

Lemma 3. Every feasible configuration contains E.

Proof. All pairs in E have Hamming distance at least b. If one lacks its direct dynamic edge, any route for that demand costs at least $\frac{3}{2}$: an all-static route is too long, a one-dynamic route needs at least one static edge, and a two-dynamic simple route needs a nonempty static segment. Every other forced demand still costs at least $\frac{1}{2}$. The missing pair therefore adds at least α to the unavoidable baseline $\alpha\frac{q}{2}$. But

$$n\left(r+\frac{3}{2}\right)-k<\alpha \tag{8}$$

is the entire allowance above that baseline. Nonnegative remaining demands cannot compensate for the excess. □

Lemma 4. For $u \in T$, every route to o of cost at most $r+\frac{3}{2}$ uses one dynamic edge $\{a,v\}$ with $v \in L$ and $D(u,a) \le 1$.

Proof. An all-static route has length at least $b > r+\frac{3}{2}$. A one-dynamic route has cost at least $D(u,a)+\frac{1}{2}+D(v,o)$, so its last dynamic endpoint lies in $B(o,r+1)$. If $v=o$, its mate is the remote vertex s. If v is any other forced vertex of that ball, Equation 6 makes its mate too far from u. The only remaining possibility is a slot, at distance r from o; then $D(u,a)\le 1$.

A two-dynamic route must finish with the forced edge $\{s,o\}$. Its first dynamic endpoint is the mate of u. By Lemma 2, every point of $B(s,r)$ already has a mate outside T, so the mate of u cannot belong to that ball. This route costs at least

$$\frac{1}{2}+(r+1)+\frac{1}{2}=r+2>r+\frac{3}{2}. \tag{9}$$

This excludes the additional route shape. □

Let a be the number of source vertices matched directly to slots. Lemma 4 implies that each of these demands costs exactly $r+\frac{1}{2}$, while every other low demand costs at least $r+\frac{3}{2}$. The half-integer length of one-dynamic routes and the larger bounds for other route shapes exclude an intermediate value. Thus

$$\text{low workload} \ge n\left(r+\frac{3}{2}\right)-a, \quad a\le k. \tag{10}$$

The budget in Equation 5 forces $a=k$ and equality for every remaining demand. All k slots are consequently occupied by source vertices. Applying Lemma 4 to each other demand shows that it is adjacent to one of those vertices.

The recovery algorithm recomputes L and returns

$$X=\{u\in T : m(u)\in L\}. \tag{11}$$

This is a dominating set of size k. The argument assumes only feasibility, not optimality, a prescribed matching outside the slots, or a canonical witness. It proves the NO direction of the Karp reduction and the recovery guarantee for every valid target witness.

5. Resource bounds and consequences

Let $N=2^d$ be the source activity-vector length. The target cube has

$$V=2^\ell=4096N^9. \tag{12}$$

Each ball enumeration visits at most V labels, taking conservatively $O(\ell^2)$ bit operations per label. A constant number of these enumerations, together with sorting, takes $O(V\ell^2)$ bit operations. The number of demands is $O(V)$; labels, weights and K have $O(\log N)$ bits. The sparse output has $O(V\log N)$ bits. Writing every cube edge requires $O(V\ell^2)$ bits and time; a dense demand matrix also

has polynomial size. The forward map and explicit expansion therefore admit the bound $O(N^9 \log^2 N)$, in addition to reading the source encoding, including an arbitrarily long encoded k.

Recovery recomputes a finite sphere and scans the supplied matching. It takes polynomial time in the total source and witness length and returns at most N indices, using $O(N \log N)$ bits. Neither map calls a solver or uses randomness. Combined with source hardness and target membership in NP, this proves Theorem 1.

The contribution is the occupied partner ball that excludes two-dynamic-edge shortcuts while preserving the domination threshold. The construction follows the occupied-region and slot framework of [1]; it does not claim that framework as new. The two-alternation result leaves the one-alternation case unresolved. Its large polynomial expansion also limits any interpretation as a practical routing method.

References

[1] D. C. Kutner and I. A. Stewart. *Reconfigurable routing in data center networks.* Theoretical Computer Science 1038 (2025), 115154. doi:10.1016/j.tcs.2025.115154. Full preprint: arXiv:2401.13359. Theorem numbers and detailed proof comparisons in this manuscript refer to that full preprint: Theorems 4, 7 and 8; Corollary 5; Sections 3.1 and 4.

A Verification and reproducibility

Finite checks support implementation fidelity; they do not replace the general proof. The prepared independent Z3 oracle checks nine source instances and ten recovery calls. A separately implemented finite-route Boolean oracle checks all 36 nonempty active-set instances in source dimensions one and two, with budgets from one through the active-set size. That suite makes 124 solver queries and 66 recovery calls, including additional unused matching edges. Both programs execute the actual forward and recovery maps in fresh processes.

The symbolic oracle uses bit-vector matching endpoints and exact workload arithmetic of sufficient width. The second oracle enumerates short routes and allows longer routes an optimistic overflow cost. An infeasible relaxation certifies NO; a feasible assignment is accepted only if no overflow route is used and the resulting matching passes direct workload evaluation. Increasing the finite cutoff to the residual budget makes this procedure exact. This is a testing method, not a claimed polynomial-time target solver.

Tiny exhaustive-matching crosschecks cover both oracles. A separate example forces a strict two-dynamic-edge optimum and tests thresholds immediately below and at its cost. Malformed objects, malformed JSON, deliberately wrong subprocess maps, invalid recovered witnesses, and a 5001-digit source budget are also checked. An independent reviewer tested a dimension-three source with 102374 forced edges and recovered a dominating set from a matching containing an additional edge between two unselected source vertices.

The manuscript's novelty comparison is to the full preprint of [1], corroborated by its 2025 journal metadata and the author's thesis. Searches and independent review on 16 September 2026 found no superseding resolution within their recorded coverage. Direct full-journal-PDF access returned 403, so no complete final-version comparison is claimed. Overlooked or unpublished results remain possible; this draft awaits human expert assessment and has not been submitted.

A.1 Reproduction

The recorded environment uses Python 3.14.7, Z3 5.1.0 and Typst 0.15.1. From the campaign root, run the finite suites and both command modes as follows:

```
python3 work/check.py --self-test
python3 work/check.py --candidate work/algorithm.py
python3 work/verify.py --candidate work/algorithm.py
python3 work/evidence/preparation/check-foundation.py
python3 work/evidence/preparation/transport-check.py
python3 work/algorithm.py < source.json
python3 work/algorithm.py --extract < recovery-request.json
typst compile work/manuscript.typ work/manuscript.pdf
```

The JSON schemas are in `work/contract.md`; the injected sources are in `work/cases.json`. Raw outputs are in `rounds/001/`, additional foundation checks in `work/evidence/preparation/`, and independent checks and the review in `reviews/initial/`. The target cube is represented by its unary dimension in these tests; its full vertex and edge sets are not materialized.